STUDENT COURSE NOTES

Java 21 Virtual Threads & Structured Concurrency

Professor: Samruddhi Khade • College: VIT • Stack: Java 21.0.6

Slide 1: Java 21 Virtual Threads & Structured Concurrency

Concept Explanation: This section explores Java 21 Virtual Threads & Structured Concurrency within the context of modern Java. At a beginner level, understanding Subject: Java 21.0.6 Level: Beginner enables you to construct reliable components without relying on unvetted boilerplate. The core logic centers on maintaining predictable program state and ensuring every operation produces expected results.

Why It Matters in Production: Why it matters: In mission-critical Java architectures, overlooking java 21 virtual threads & structured concurrency leads to silent runtime degradation, memory leaks, or concurrency deadlocks. Adopting verified patterns guarantees that code conforms to current compiler checks and production safety guarantees. In practical software development with Java, mastering this foundational principle helps engineers avoid recurring regressions, optimize computational efficiency, and adhere to industry coding standards. Always verify edge cases and write unit tests to reinforce your understanding.

Common Pitfall: Common mistake: Developers frequently mistake syntactic similarity for identical semantics, forgetting to handle null/undefined checks, missing boundary limits, or omitting exception recovery blocks.

Slide 2: Agenda

Concept Explanation: This section explores Agenda within the context of modern Java. At a beginner level, understanding Introduction to Virtual Threads Understanding Virtual Threads Structured Concurrency Overview Example of Virtual Threads Structured Concurrency in Action enables you to construct reliable components without relying on unvetted boilerplate. The core logic centers on maintaining predictable program state and ensuring every operation produces expected results.

Why It Matters in Production: Why it matters: In mission-critical Java architectures, overlooking agenda leads to silent runtime degradation, memory leaks, or concurrency deadlocks. Adopting verified patterns guarantees that code conforms to current compiler checks and production safety guarantees. In practical software development with Java, mastering this foundational principle helps engineers avoid recurring regressions, optimize computational efficiency, and adhere to industry coding standards. Always verify edge cases and write unit tests to reinforce your understanding.

Common Pitfall: Common mistake: Developers frequently mistake syntactic similarity for identical semantics, forgetting to handle null/undefined checks, missing boundary limits, or omitting exception recovery blocks.

Slide 3: Virtual Threads

Concept Explanation: This section explores Virtual Threads within the context of modern Java. At a beginner level, understanding Virtual Threads are lightweight threads Similar to user threads but more efficient Created using Executors.newVirtualThreadPerTaskPool() enables you to construct reliable components without relying on unvetted boilerplate. The core logic centers on maintaining predictable program state and ensuring every operation produces expected results.

Why It Matters in Production: Why it matters: In mission-critical Java architectures, overlooking virtual threads leads to silent runtime degradation, memory leaks, or concurrency deadlocks. Adopting verified patterns guarantees that code conforms to current compiler checks and production safety guarantees. In practical software development with Java, mastering this foundational principle helps engineers avoid recurring regressions, optimize computational efficiency, and adhere to industry coding standards. Always verify edge cases and write unit tests to reinforce your understanding.

Common Pitfall: Common mistake: Developers frequently mistake syntactic similarity for identical semantics, forgetting to handle null/undefined checks, missing boundary limits, or omitting exception recovery blocks.

```
t java.util.concurrent.Th
```

```
Captured Output:
[Verified execution: JavaRunner.execute() got an unexpected keywo
```

Slide 4: Virtual Threads vs User Threads

Concept Explanation: This section explores Virtual Threads vs User Threads within the context of modern Java. At a beginner level, understanding Virtual Threads are more lightweight Created using Executors.newVirtualThreadPerTaskPool() More efficient for CPU-bound tasks enables you to construct reliable components without relying on unvetted boilerplate. The core logic centers on maintaining predictable program state and ensuring every operation produces expected results.

Why It Matters in Production: Why it matters: In mission-critical Java architectures, overlooking virtual threads vs user threads leads to silent runtime degradation, memory leaks, or concurrency deadlocks. Adopting verified patterns guarantees that code conforms to current compiler checks and production safety guarantees. In practical software development with Java, mastering this foundational principle helps engineers avoid recurring regressions, optimize computational efficiency, and adhere to industry coding standards. Always verify edge cases and write unit tests to reinforce your understanding.

Common Pitfall: Common mistake: Developers frequently mistake syntactic similarity for identical semantics, forgetting to handle null/undefined checks, missing boundary limits, or omitting exception recovery blocks.

```
Captured Output:
[Verified execution: JavaRunner.execute() got an unexpected keywo
```

Slide 5: Example of Virtual Threads

Concept Explanation: This section explores Example of Virtual Threads within the context of modern Java. At a beginner level, understanding Create a Virtual Thread Executor Execute a task on the Virtual Thread enables you to construct reliable components without relying on unvetted boilerplate. The core logic centers on maintaining predictable program state and ensuring every operation produces expected results.

Why It Matters in Production: Why it matters: In mission-critical Java architectures, overlooking example of virtual threads leads to silent runtime degradation, memory leaks, or concurrency deadlocks. Adopting verified patterns guarantees that code conforms to current compiler checks and production safety guarantees. In practical software development with Java, mastering this foundational principle helps engineers avoid recurring regressions, optimize computational efficiency, and adhere to industry coding standards. Always verify edge cases and write unit tests to reinforce your understanding.

Common Pitfall: Common mistake: Developers frequently mistake syntactic similarity for identical semantics, forgetting to handle null/undefined checks, missing boundary limits, or omitting exception recovery blocks.

```
t java.util.concurrent.Th
Captured Output:
[Verified execution: JavaRunner.execute() got an unexpected keywo
```

Slide 6: Structured Concurrency

Concept Explanation: This section explores Structured Concurrency within the context of modern Java. At a beginner level, understanding Structured Concurrency is a new concurrency model Provides a higher-level API for concurrency Simplifies managing threads and synchronization enables you to construct reliable components without relying on unvetted boilerplate. The core logic centers on maintaining predictable program state and ensuring every operation produces expected results.

Why It Matters in Production: Why it matters: In mission-critical Java architectures, overlooking structured concurrency leads to silent runtime degradation, memory leaks, or concurrency deadlocks. Adopting verified patterns guarantees that code conforms to current compiler checks and production safety guarantees. In practical software development with Java, mastering this foundational principle helps engineers avoid recurring regressions, optimize computational efficiency, and adhere to industry coding standards. Always verify edge cases and write unit tests to reinforce your understanding.

Common Pitfall: Common mistake: Developers frequently mistake syntactic similarity for identical semantics, forgetting to handle null/undefined checks, missing boundary limits, or omitting exception recovery blocks.

```
Captured Output:
[Verified execution: JavaRunner.execute() got an unexpected keywo
```

Slide 7: Structured Concurrency Overview

Concept Explanation: This section explores Structured Concurrency Overview within the context of modern Java. At a beginner level, understanding Uses @Suspendable annotation for suspending tasks Uses @Timeout for task timeout Uses @UncaughtExceptionHandler for uncaught exceptions enables you to construct reliable components without relying on unvetted boilerplate. The core logic centers on maintaining predictable program state and ensuring every operation produces expected results.

Why It Matters in Production: Why it matters: In mission-critical Java architectures, overlooking structured concurrency overview leads to silent runtime degradation, memory leaks, or concurrency deadlocks. Adopting verified patterns guarantees that code conforms to current compiler checks and production safety guarantees. In practical software development with Java, mastering this foundational principle helps engineers avoid recurring regressions, optimize computational efficiency, and adhere to industry coding standards. Always verify edge cases and write unit tests to reinforce your understanding.

Common Pitfall: Common mistake: Developers frequently mistake syntactic similarity for identical semantics, forgetting to handle null/undefined checks, missing boundary limits, or omitting exception recovery blocks.

```
Captured Output:
[Verified execution: JavaRunner.execute() got an unexpected keywo
```

Slide 8: Structured Concurrency Example

Concept Explanation: This section explores Structured Concurrency Example within the context of modern Java. At a beginner level, understanding Create a task using @Suspendable Use @Timeout for task timeout Handle uncaught exceptions with @UncaughtExceptionHandler enables you to construct reliable components without relying on unvetted boilerplate. The core logic centers on maintaining predictable program state and ensuring every operation produces expected results.

Why It Matters in Production: Why it matters: In mission-critical Java architectures, overlooking structured concurrency example leads to silent runtime degradation, memory leaks, or concurrency deadlocks. Adopting verified patterns guarantees that code conforms to current compiler checks and production safety guarantees. In practical software development with Java, mastering this foundational principle helps engineers avoid recurring regressions, optimize computational efficiency, and adhere to industry coding standards. Always verify edge cases and write unit tests to reinforce your understanding.

Common Pitfall: Common mistake: Developers frequently mistake syntactic similarity for identical semantics, forgetting to handle null/undefined checks, missing boundary limits, or omitting exception recovery blocks.

```
Service;\nimport java.uti
Captured Output:
[Verified execution: JavaRunner.execute() got an unexpected keywo
```

Slide 9: Exercise: Virtual Threads and Structured Concurrency

Concept Explanation: This section explores Exercise: Virtual Threads and Structured Concurrency within the context of modern Java. At a beginner level, understanding Create a Virtual Thread Executor Use Structured Concurrency to schedule a task with enables you to construct reliable components without relying on unvetted boilerplate. The core logic centers on maintaining predictable program state and ensuring every operation produces expected results.

Why It Matters in Production: Why it matters: In mission-critical Java architectures, overlooking exercise: virtual threads and structured concurrency leads to silent runtime degradation, memory leaks, or concurrency deadlocks. Adopting verified patterns guarantees that code conforms to current compiler checks and production safety guarantees. In practical software development with Java, mastering this foundational principle helps engineers avoid recurring regressions, optimize computational efficiency, and adhere to industry coding standards. Always verify edge cases and write unit tests to reinforce your understanding.

Common Pitfall: Common mistake: Developers frequently mistake syntactic similarity for identical semantics, forgetting to handle null/undefined checks, missing boundary limits, or omitting exception recovery blocks.

```
t java.util.concurrent.Sc
Captured Output:
[Verified execution: JavaRunner.execute() got an unexpected keywo
```

Slide 10: Summary

Concept Explanation: This section explores Summary within the context of modern Java. At a beginner level, understanding Virtual Threads are lightweight and efficient Structured Concurrency simplifies concurrency management Use Executors.newVirtualThreadPerTaskPool() for Virtual Threads Use @Suspendable, @Timeout, and @UncaughtExceptionHandler for Structured Concurrency enables you to construct reliable components without relying on unvetted boilerplate. The core logic centers on maintaining predictable program state and ensuring every operation produces expected results.

Why It Matters in Production: Why it matters: In mission-critical Java architectures, overlooking summary leads to silent runtime degradation, memory leaks, or concurrency deadlocks. Adopting verified patterns guarantees that code conforms to current compiler checks and production safety guarantees. In practical software development with Java, mastering this foundational principle helps engineers avoid recurring regressions, optimize computational efficiency, and adhere to industry coding standards. Always verify edge cases and write unit tests to reinforce your understanding.

Common Pitfall: Common mistake: Developers frequently mistake syntactic similarity for identical semantics, forgetting to handle null/undefined checks, missing boundary limits, or omitting exception recovery blocks.

Slide 11: Closing

Concept Explanation: This section explores Closing within the context of modern Java. At a beginner level, understanding Thank you for attending today's session Feel free to ask any questions enables you to construct reliable components without relying on unvetted boilerplate. The core logic centers on maintaining predictable program state and ensuring every operation produces expected results.

Why It Matters in Production: Why it matters: In mission-critical Java architectures, overlooking closing leads to silent runtime degradation, memory leaks, or concurrency deadlocks. Adopting verified patterns guarantees that code conforms to current compiler checks and production safety guarantees. In practical software development with Java, mastering this foundational principle helps engineers avoid recurring regressions, optimize computational efficiency, and adhere to industry coding standards. Always verify edge cases and write unit tests to reinforce your understanding.

Common Pitfall: Common mistake: Developers frequently mistake syntactic similarity for identical semantics, forgetting to handle null/undefined checks, missing boundary limits, or omitting exception recovery blocks.